

SISTEMAS DISTRIBUIDOS

PRÁCTICA 5

Mexi Banco: pruebas, persistencia y análisis del código

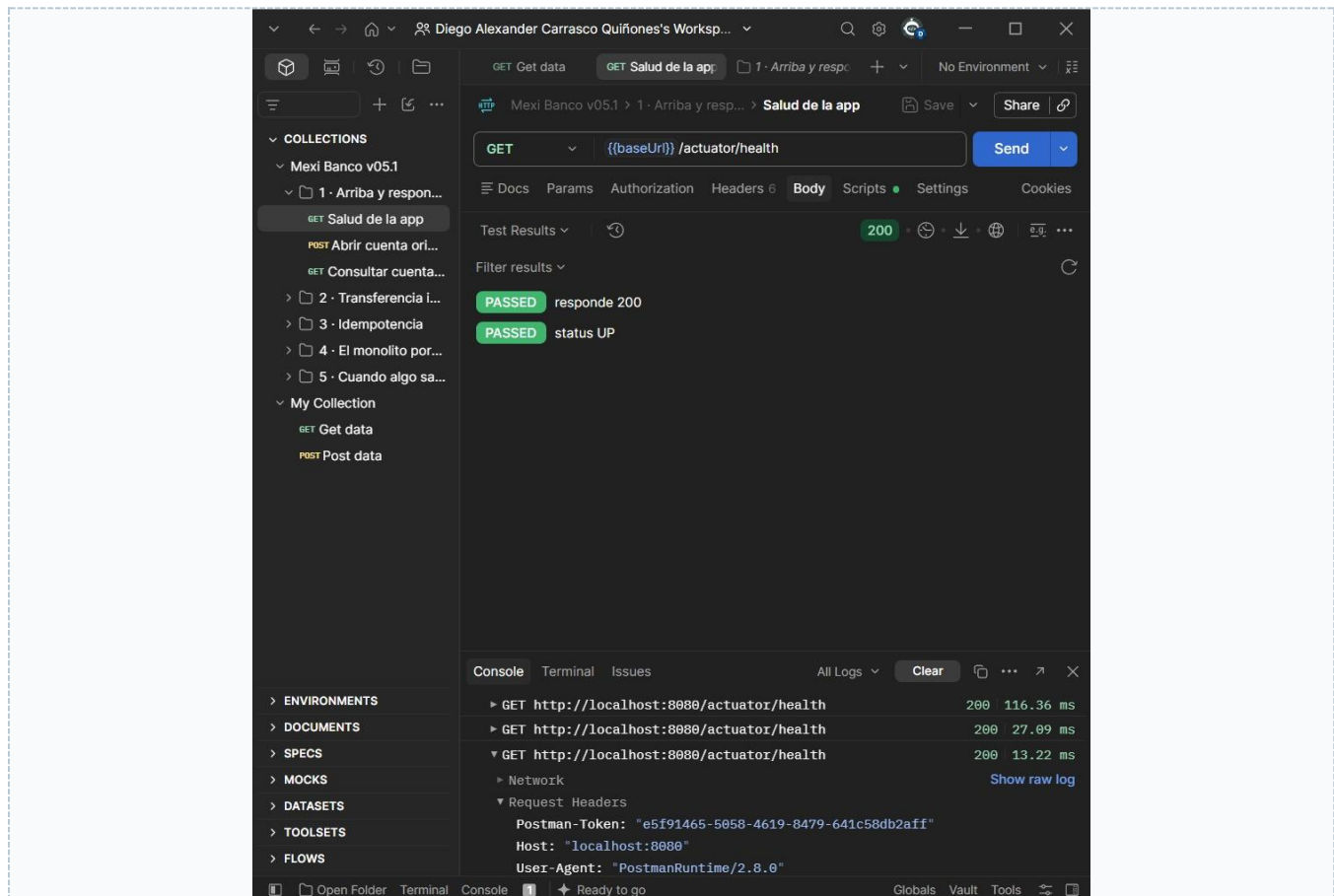
Alumno	Diego Alexander Carrasco Quiñones	Fecha entrega	27-09-2026
--------	-----------------------------------	---------------	------------

1 Pruebas de los endpoints con Postman

Tras levantar la aplicación con Docker Compose, se usó **Postman Desktop** para probar los endpoints de Mexi Banco, disponibles en <http://localhost:8080>. Se utilizaron dos cuentas (origen y destino) y las CLABEs se generaron automáticamente con una variable de colección para evitar conflictos con cuentas creadas antes.

2 Estado de la aplicación

Primera prueba: `GET /actuador/health`. Respuesta **200 OK** con `{ "status": "UP" }`, lo que indica que Spring Boot está funcionando y responde solicitudes. Se hizo antes de cualquier operación bancaria para confirmar que el servidor estuviera disponible.

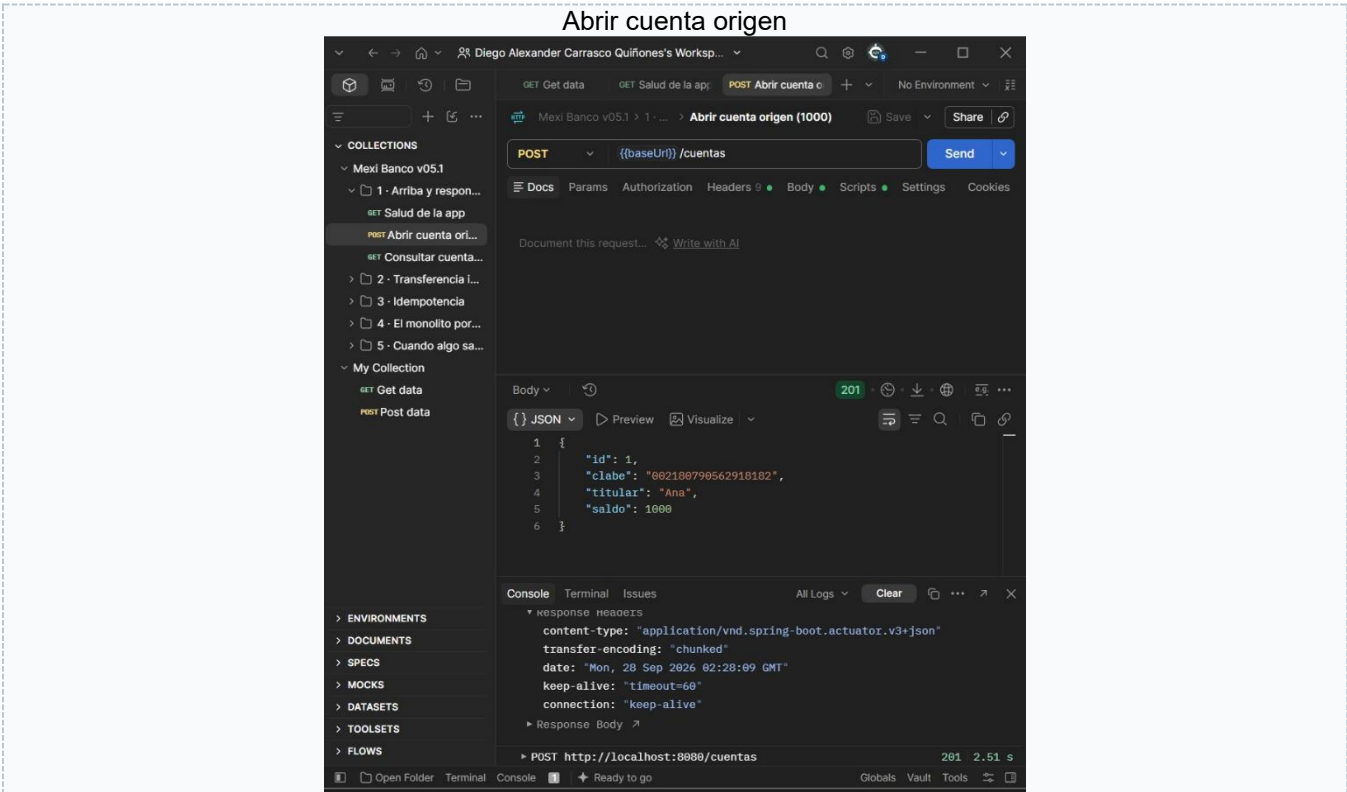


3-5 Apertura y consulta de cuentas

Cuenta origen (POST /cuentas): saldo inicial de \$1000, respuesta **201 Created**. El servicio verificó que la CLABE no existiera, creó la cuenta en PostgreSQL y, al haber saldo inicial mayor a cero, registró un movimiento tipo **DEPOSITO**.

Consulta (GET /cuentas/{clabe}): **200 OK** con saldo \$1000; CuentaService encontró la cuenta mediante el repositorio. **Cuenta destino**: creada con \$500, respuesta **201 Created**.

Cuenta	Saldo antes de transferir
Origen	\$1000
Destino	\$500



Consultar cuenta origen

The screenshot displays the Postman interface for a REST client. The top bar shows the workspace name "Diego Alexander Carrasco Quiñones's Worksp...". The left sidebar contains a "COLLECTIONS" list with "Mexi Banco v05.1" expanded, showing several endpoints. The main area is configured for a GET request to "Consultar cuenta origen" with the URL template "GET {{baseUri}} /cuentas/ {{origen}}". The "Send" button is visible. Below the URL bar, the "Query Params" section is empty. The "Body" tab is selected, showing a JSON response:

```
1 {
2   "id": 1,
3   "clabe": "002180790562918182",
4   "titular": "Ana",
5   "saldo": 1000.00
6 }
```

The "Console" tab at the bottom shows the execution results:

- POST http://localhost:8080/cuentas: 201 2.51 s
- GET http://localhost:8080/cuentas/002180790562918182: 200 191.36 ms

The "Test Results" section shows two passed tests:

- PASSED responde 200
- PASSED saldo 1000

The "Console" output also shows the response headers:

```
Content-Type: application/vnd.openapi+json; charset=utf-8
transfer-encoding: chunked
date: Mon, 28 Sep 2026 02:28:09 GMT
keep-alive: timeout=60
connection: keep-alive
```

Apertura de cuenta destino

The image displays two screenshots of the Postman application interface, showing a REST client request and its response.

Top Screenshot: The interface shows a REST client request for a POST method to the endpoint `{{baseUrl}}/cuentas`. The request body is a JSON object:

```
{  "id": 2,  "clabe": "012100790562918182",  "titular": "Beto",  "saldo": 500}
```

The response status is 201. The response body is a JSON object:

```
{  "id": 2,  "clabe": "012100790562918182",  "titular": "Beto",  "saldo": 500}
```

Bottom Screenshot: The interface shows the same REST client request, but the response status is 201. The response body is a JSON object:

```
{  "id": 2,  "clabe": "012100790562918182",  "titular": "Beto",  "saldo": 500}
```

The bottom screenshot also shows the response body in the console, indicating a successful response with status 201.

6-8 Transferencia interna de \$200

Solicitud a `POST /transferencias` desde origen hacia destino: **201 Created**. Internamente la aplicación: (1) comprueba que origen y destino sean distintas, (2) verifica que existan, (3) comprueba saldo suficiente, (4) descuenta \$200 al origen, (5) abona \$200 al destino, (6-7) registra movimientos de salida y entrada, (8) guarda la transferencia y (9) genera las notificaciones, todo dentro de una **transacción**.

Cuenta	Antes	Después
Origen	\$1000	\$800
Destino	\$500	\$700

Verificación: $\$800 + \$700 = \$1500$, igual que $\$1000 + \500 . La transferencia conservó el total de dinero. Además, `GET /transferencias/{id}` devolvió **200 OK** con monto \$200 (ID, origen, destino, monto y fecha), por lo que la operación quedó registrada de forma independiente.

Transferencia de \$200

The screenshot shows a REST client interface with the following details:

- Method:** POST
- URL:** `{{baseUrl}}/transferencias`
- Status:** 201
- Response Body (JSON):**

```
{
  "id": 1,
  "claveOrigen": "062188798562918182",
  "claveDestino": "012188798562918182",
  "monto": 200,
  "fecha": "2026-09-28T02:43:17.598676001Z"
}
```
- Console:** Shows logs for the POST request and the response body.

Saldos después de la transferencia

The image displays two screenshots of the Postman application, showing the results of API tests for account balances after a transfer operation.

Top Screenshot: Consultar origen (800)

The test is a GET request to `{{baseUrl}}/cuentas/{{origen}}`. The response status is 200. The response body is JSON:

```
{
  "id": 1,
  "clabe": "062180799562918182",
  "titular": "Ana",
  "saldo": 800.00
}
```

The console shows the following log:

```
connection: "keep-alive"
Response Body
```

Bottom Screenshot: Consultar destino (700)

The test is a GET request to `{{baseUrl}}/cuentas/{{destino}}`. The response status is 200. The response body is JSON:

```
{
  "id": 2,
  "clabe": "012180799562918182",
  "titular": "Beto",
  "saldo": 700.00
}
```

The console shows the following log:

```
Response Body
```

Both screenshots show a list of test results in the console, including POST and GET requests to various endpoints, with their respective status codes and response times.

Consulta de transferencia

The screenshot displays the Postman REST client interface for a project named 'Mexi Banco v05.1'. The top section shows a GET request to 'Consultar la transferencia' with a 200 status code and a JSON response. The bottom section shows the same interface with test results for 'responde 200' and 'monto 200' both passing.

Request Details:

- Method: GET
- URL: `{{baseUrl}}/transferencias/{{transferenciaId}}`
- Query Params: Key, Value, Description

Response Body (JSON):

```
{
  "id": 1,
  "claveOrigen": "062188798562918182",
  "claveDestino": "012188798562918182",
  "monto": 200.00,
  "fecha": "2026-09-28T02:43:17.598676Z"
}
```

Test Results:

- PASSED responde 200
- PASSED monto 200

Console Log:

- POST http://localhost:8080/cuentas 201 2.51 s
- GET http://localhost:8080/cuentas/062188798562918182 200 191.36 ms
- POST http://localhost:8080/cuentas 201 31.95 ms
- POST http://localhost:8080/transferencias 201 520.16 ms
- GET http://localhost:8080/cuentas/062188798562918182 200 186.74 ms
- GET http://localhost:8080/cuentas/012188798562918182 200 23.16 ms
- GET http://localhost:8080/transferencias/1 200 313.91 ms

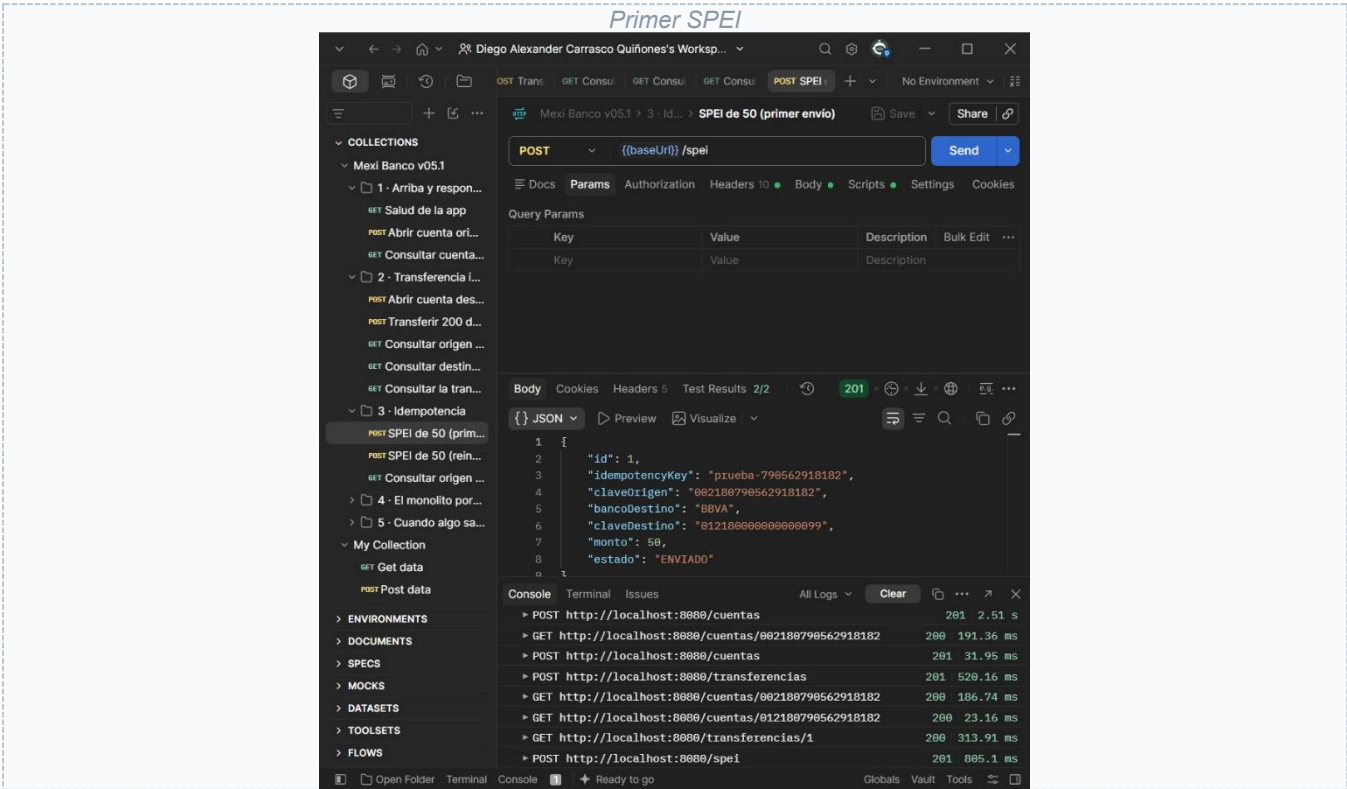
9-10 Operación SPEI e idempotencia

Se envió un SPEI de \$50 (origen con \$800) usando una cabecera `Idempotency-Key`. Respuesta **201 Created**, estado `ENVIADO`. La aplicación registró la solicitud, verificó que la clave no se hubiera usado, descontó \$50, registró un movimiento `SPEI_ENVIADO`, cambió el estado a `ENVIADO` y generó una notificación. Saldo: \$800 a \$750.

Reintento con la misma clave: la respuesta fue de nuevo 201 Created con el mismo identificador SPEI, pero **no se volvió a descontar**; el saldo siguió en \$750.

Primera solicitud	Reintento (misma Idempotency-Key)
Crear SPEI, descontar \$50, quedar en ENVIADO	La clave ya existe, se devuelve la operación existente y no se descuenta de nuevo

¿Por qué importa? Un cliente podría repetir una solicitud porque una falla de red le impidió recibir la respuesta. Sin idempotencia habría un segundo cargo; la clave permite reconocer que ambas solicitudes son la misma operación.



Reintento con la misma Idempotency-Key

The screenshot shows a Postman interface with a collection named 'Mexi Banco v05.1'. The selected request is 'POST SPEI de 50 (reintento, misma clave)'. The URL is set to 'POST {{baseUrl}} /spei'. The 'Send' button is highlighted. The 'Test Results' tab shows two passed tests: 'responde 201' and 'es el MISMO SPEI, no uno nuevo'. The console log shows the request was successful with a 201 status code.

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body Cookies Headers 5 Test Results 2/2 201

Filter results

PASSED responde 201

PASSED es el MISMO SPEI, no uno nuevo

Console Terminal Issues All Logs Clear

- GET http://localhost:8080/cuentas/002180790562918182 200 191.36 ms
- POST http://localhost:8080/cuentas 201 31.95 ms
- POST http://localhost:8080/transferencias 201 520.16 ms
- GET http://localhost:8080/cuentas/002180790562918182 200 186.74 ms
- GET http://localhost:8080/cuentas/012180790562918182 200 23.16 ms
- GET http://localhost:8080/transferencias/1 200 313.91 ms
- POST http://localhost:8080/spei 201 805.1 ms
- POST http://localhost:8080/spei 201 543.68 ms

Consultar origen (750, no 700)

The screenshot shows a Postman interface with a collection named 'Mexi Banco v05.1'. The selected request is 'Consultar origen (750, no 700)'. The URL is set to 'GET {{baseUrl}} /cuentas/{{origen}}'. The 'Send' button is highlighted. The 'Test Results' tab shows two passed tests: 'responde 200' and 'el saldo es 750.00'. The console log shows the request was successful with a 200 status code.

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body Cookies Headers 5 Test Results 2/2 200

JSON Preview Visualize

```

1 {
2   "id": 1,
3   "clabe": "002180790562918182",
4   "titular": "Ana",
5   "saldo": 750.00
6 }
```

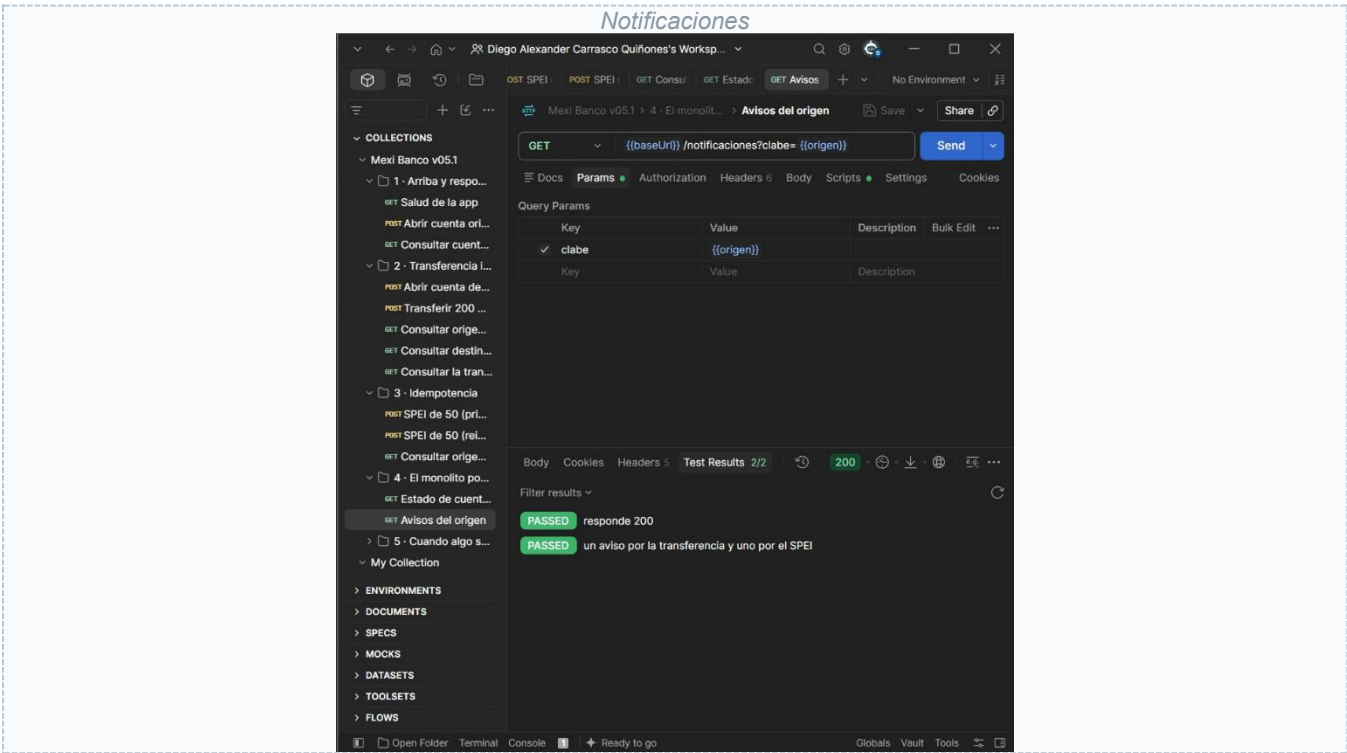
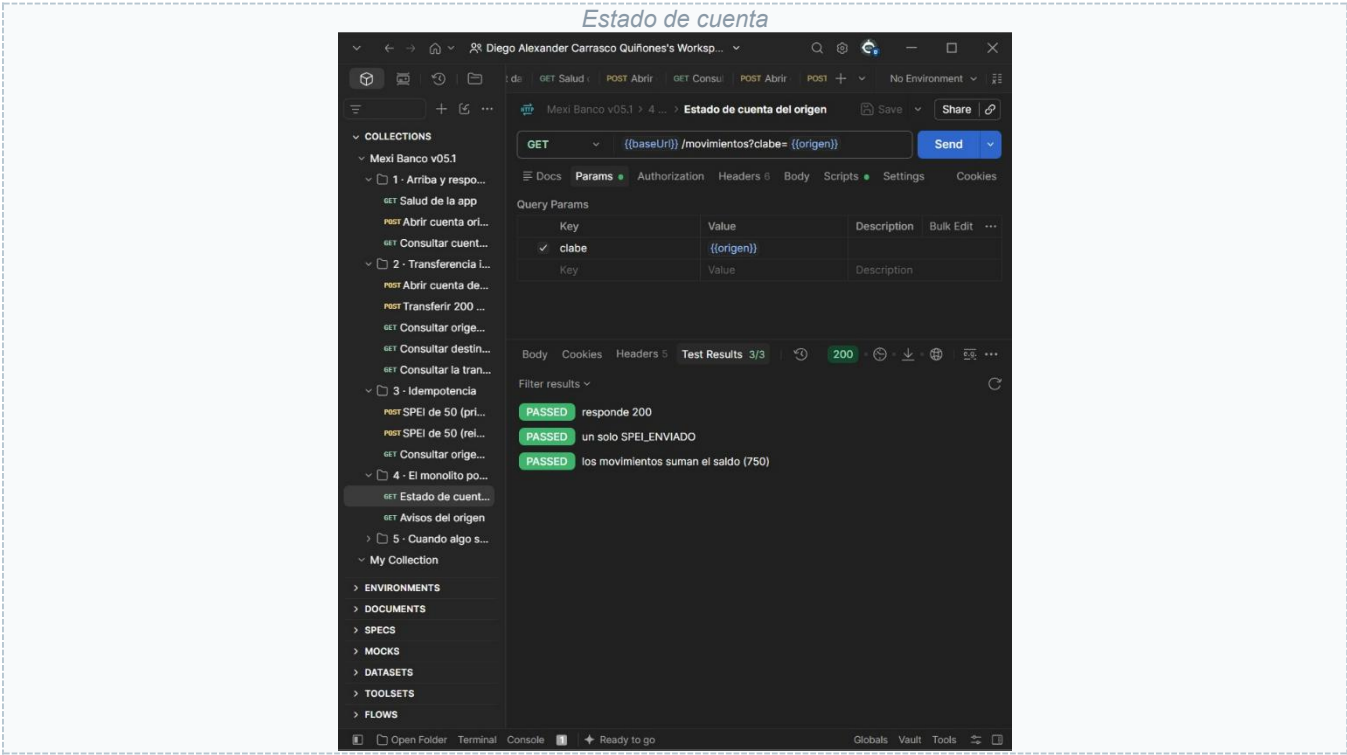
Console Terminal Issues All Logs Clear

- POST http://localhost:8080/cuentas 201 31.95 ms
- POST http://localhost:8080/transferencias 201 520.16 ms
- GET http://localhost:8080/cuentas/002180790562918182 200 186.74 ms
- GET http://localhost:8080/cuentas/012180790562918182 200 23.16 ms
- GET http://localhost:8080/transferencias/1 200 313.91 ms
- POST http://localhost:8080/spei 201 805.1 ms
- POST http://localhost:8080/spei 201 543.68 ms
- GET http://localhost:8080/cuentas/002180790562918182 200 88.12 ms

11-12 Estado de cuenta y notificaciones

GET /movimientos?clabe=... devolvió **200 OK** con los movimientos, entre ellos `SPEI_ENVIADO` y los de la transferencia. Saldo final de la cuenta origen: **\$750** (\$1000 - \$200 = \$800; \$800 - \$50 = \$750).

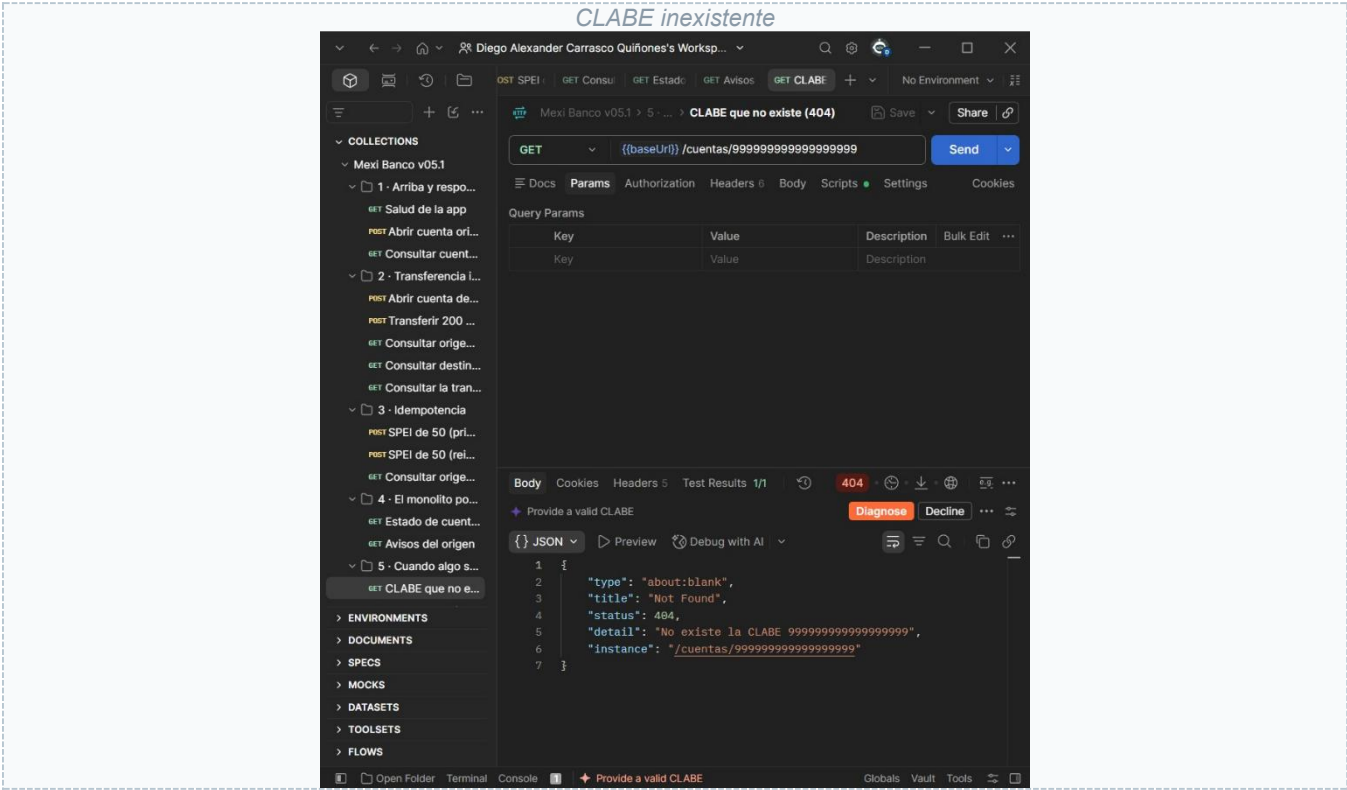
GET /notificaciones?clabe=... devolvió **200 OK** con dos notificaciones, una de la transferencia y otra del SPEI. Cada movimiento importante puede generar una notificación que el sistema conserva como registro.

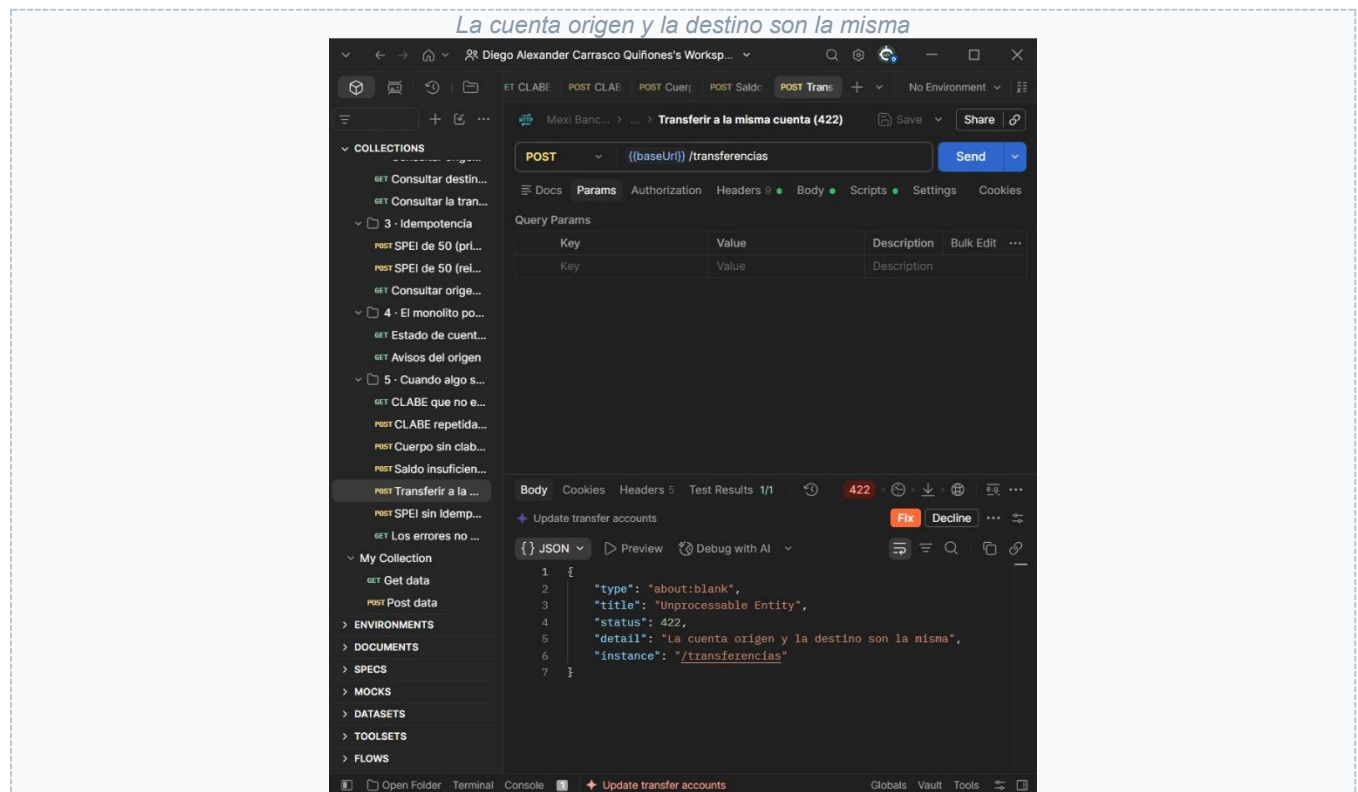
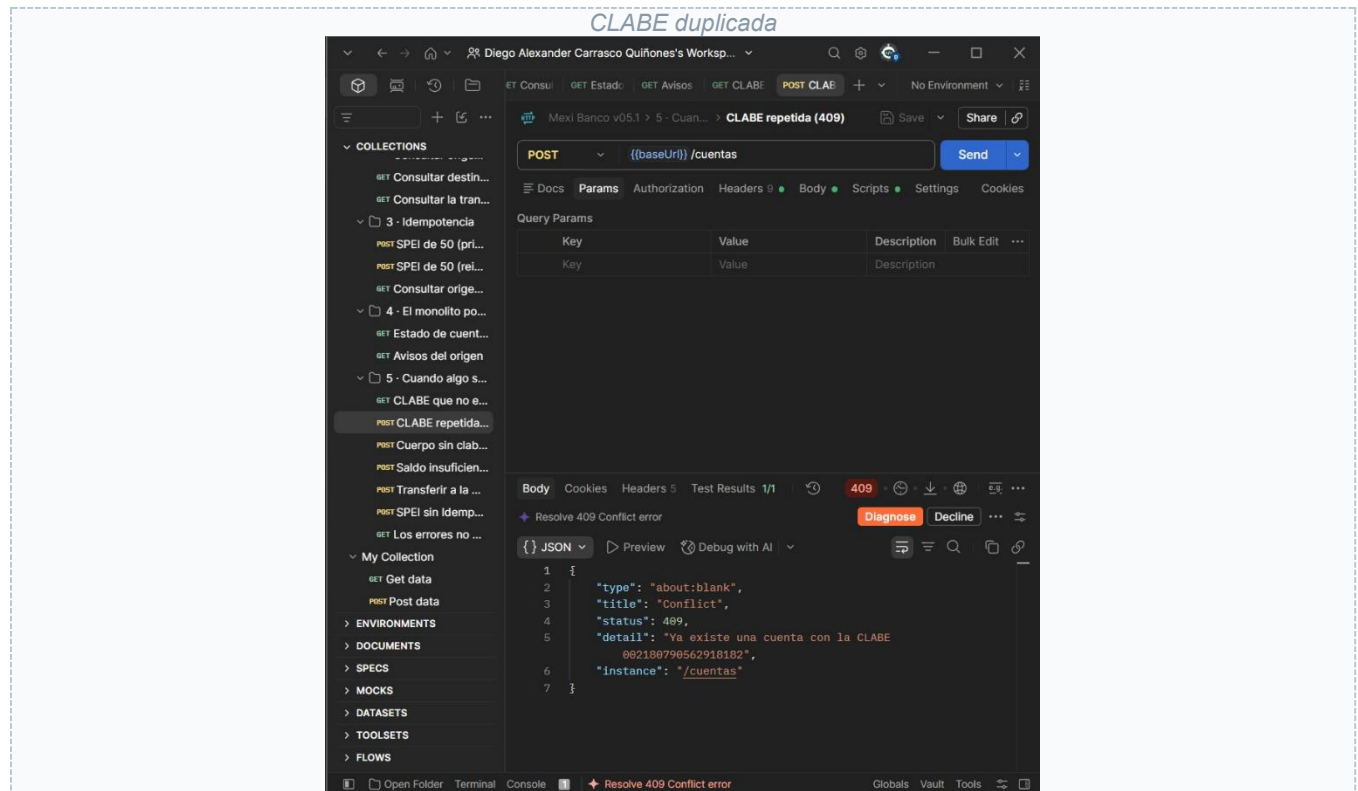


13-19 Pruebas de errores y validaciones

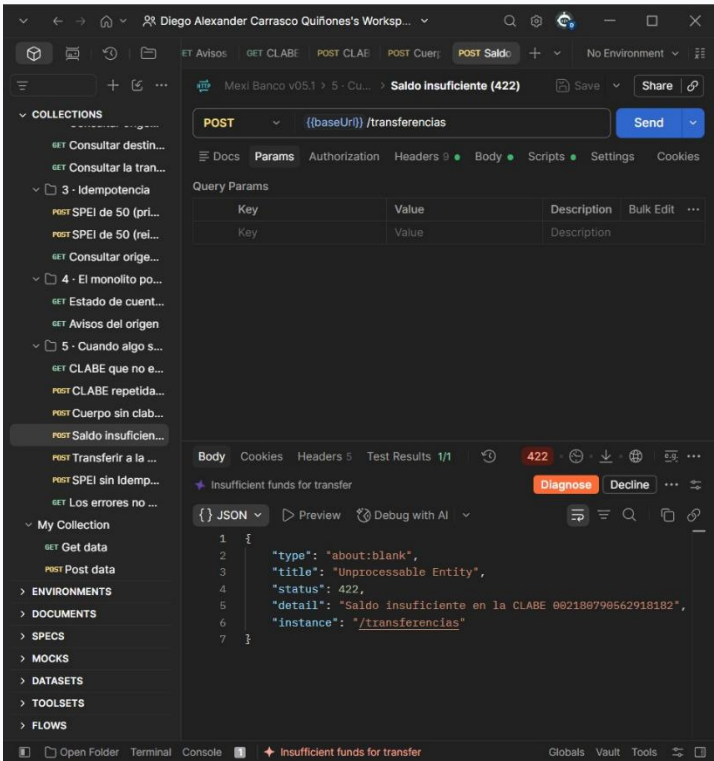
Prueba	Código	Qué ocurrió
CLABE inexistente	404 Not Found	<code>CuentaService</code> no encontró la cuenta y lanzó una excepción de recurso no encontrado; no se modificó nada.
CLABE duplicada	409 Conflict	El servicio comprueba si la CLABE ya existe; además la entidad <code>Cuenta</code> tiene <code>unique = true</code> sobre la CLABE.
Falta el campo clabe	400 Bad Request	Validaciones como <code>@NotBlank</code> rechazan la solicitud antes de tocar la base de datos.
Saldo insuficiente	422 Unprocessable	Si <code>saldo < monto</code> se rechaza la operación; la cuenta no queda en negativo y el saldo no cambia.
Transferir a la misma cuenta	422 Unprocessable	El servicio comprueba explícitamente que origen y destino sean diferentes.
SPEI sin Idempotency-Key	400 Bad Request	La clave es obligatoria; sin ella no se garantiza el comportamiento idempotente y no se descuenta dinero.

Verificación final: tras todas las pruebas de error, el saldo de la cuenta origen seguía en **\$750**. Las solicitudes rechazadas no modificaron nada, por lo que los errores de negocio se manejaron correctamente.





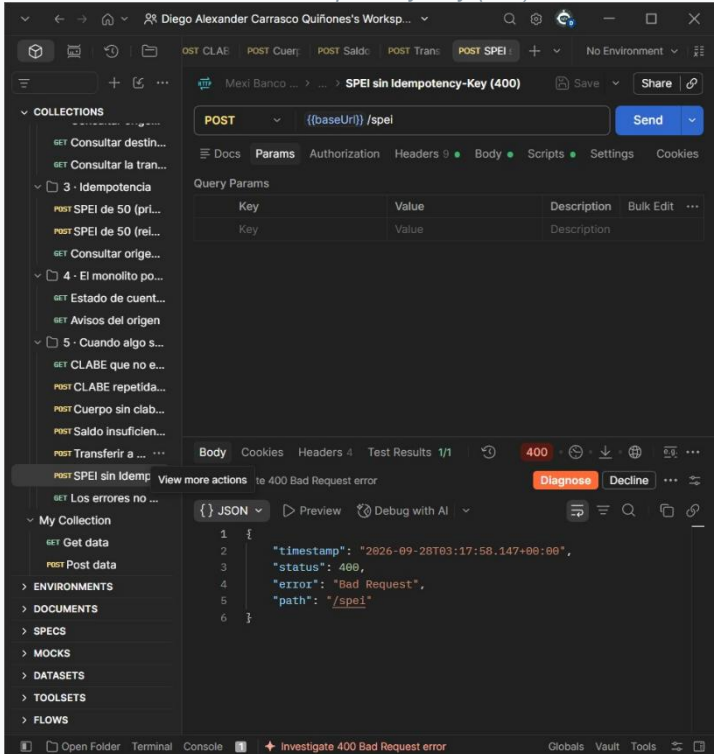
Saldo insuficiente



The screenshot shows a Postman interface with a collection named 'Mexi Banco v05.1'. The selected request is 'POST Saldo insuficiente (422)' with a URL of 'POST {{baseUrl}}/transferencias'. The response status is 422. The JSON body of the response is as follows:

```
1 {
2   "type": "about:blank",
3   "title": "Unprocessable Entity",
4   "status": 422,
5   "detail": "Saldo insuficiente en la CLABE 002180790562918182",
6   "instance": "/transferencias"
7 }
```

SPEI sin idempotency-Key (400)



The screenshot shows a Postman interface with a collection named 'Mexi Banco v05.1'. The selected request is 'POST SPEI sin Idempotency-Key (400)' with a URL of 'POST {{baseUrl}}/spei'. The response status is 400. The JSON body of the response is as follows:

```
1 {
2   "timestamp": "2026-09-28T03:17:50.147+00:00",
3   "status": 400,
4   "error": "Bad Request",
5   "path": "/spei"
6 }
```

20-24 Persistencia con Docker Compose

Con la cuenta origen en \$750 se probó el ciclo de vida de los datos. `docker compose down` eliminó los contenedores app y db y la red, pero **no** el volumen `mexibanco_datos`. Al ejecutar `docker compose up -d`, PostgreSQL se reconectó al mismo volumen y la cuenta seguía existiendo con \$750.

Después se probó `docker compose down -v`: la opción `-v` también elimina los volúmenes (se eliminaron app, db, el volumen y la red). Al levantar de nuevo, el volumen era nuevo y estaba vacío, y la consulta de la cuenta anterior devolvió **404 Not Found**. Esto confirmó que el volumen es el responsable de conservar los datos.

Acción	Contenedor	Volumen	Datos
<code>down</code>	Se elimina	Se conserva	Se conservan
<code>up -d</code>	Se recrea	Se reutiliza	Continúan
<code>down -v</code>	Se elimina	Se elimina	Se pierden
<code>up -d</code>	Se recrea	Se crea uno nuevo	Los anteriores no existen

Conclusión: eliminar el contenedor no es lo mismo que eliminar los datos. El volumen tiene un ciclo de vida independiente del contenedor.

25-29 Análisis de los módulos

La aplicación sigue la estructura **Controller, Service, Repository, Base de datos**, separando la comunicación HTTP, las reglas de negocio y el acceso a los datos.

Módulo	Componentes	Responsabilidad
cuenta	<code>Cuenta</code> , <code>CuentaController</code> , <code>CuentaService</code> , <code>CuentaRepository</code>	Administra cuentas y saldos. <code>Cuenta</code> guarda id, clabe (única y obligatoria), titular, saldo (<code>BigDecimal</code>) y version (<code>@Version</code> , concurrencia optimista). Endpoints <code>POST /cuentas</code> y <code>GET /cuentas/{clabe}</code> . El servicio ofrece abrir, consultar, cargar (descuenta y valida saldo) y abonar; otros módulos lo usan en vez de acceder al repositorio. El repositorio usa consultas derivadas (<code>findByClabe</code> , <code>existsByClabe</code>).
	Cuenta — Entidad JPA que representa una cuenta bancaria. Guarda id, clabe, titular, saldo y version. La CLABE es única y obligatoria, el saldo utiliza <code>BigDecimal</code> y <code>@Version</code> permite control de concurrencia optimista.	
	CuentaController — Expone los endpoints <code>POST /cuentas</code> para abrir una cuenta y <code>GET /cuentas/{clabe}</code> para consultar una cuenta. También valida los datos recibidos.	
	CuentaService — Contiene las reglas de negocio para abrir, consultar, cargar y abonar cuentas. Valida que no exista una CLABE duplicada y que exista saldo suficiente antes de realizar un cargo .	
	CuentaRepository — Accede a la base de datos mediante Spring Data JPA. Utiliza consultas derivadas como <code>findByClabe()</code> y <code>existsByClabe()</code> .	
movimiento	<code>Movimiento</code> , <code>MovimientoController</code> , <code>MovimientoService</code> , <code>TipoMovimiento</code>	Historial de operaciones: id, clabeCuenta, tipo, monto, saldoResultante (saldo tras el movimiento), fecha y referencia. Tipos: <code>DEPOSITO</code> , <code>TRANSFERENCIA ENVIADA</code> , <code>TRANSFERENCIA RECIBIDA</code> , <code>SPEI_ENVIADO</code> . <code>GET /movimientos?clabe=</code> los muestra del más reciente al más antiguo.
	Movimiento — Entidad que representa un movimiento realizado sobre una cuenta. Guarda id, clabeCuenta, tipo, monto, saldoResultante, fecha y referencia.	

Módulo	Componentes	Responsabilidad
	MovimientoController — Expone GET /movimientos?clabe=... para consultar el historial de movimientos de una cuenta.	
	MovimientoService — Registra nuevos movimientos y consulta el historial. Recibe el saldo resultante después de una operación para conservar el estado que tenía la cuenta en ese momento.	
	MovimientoRepository — Consulta y almacena movimientos mediante JPA. Utiliza findByClabeCuentaOrderByFechaDesc() para obtenerlos del más reciente al más antiguo.	
	TipoMovimiento — Define los tipos permitidos: DEPOSITO, TRANSFERENCIA_ENVIADA, TRANSFERENCIA_RECIBIDA y SPEI_ENVIADO.	
transferencia	Transferencia, TransferenciaController, TransferenciaService	Coordina las transferencias internas (id, origen, destino, monto, fecha) con POST /transferencias y GET /transferencias/{id}. Ejecuta los 9 pasos vistos antes en una transacción local.
	Transferencia — Entidad que representa una transferencia interna completa. Guarda id, claveOrigen, claveDestino, monto y fecha.	
	TransferenciaController — Expone POST /transferencias para realizar una transferencia y GET /transferencias/{id} para consultar una transferencia específica. Valida origen, destino y monto.	
	TransferenciaService — Coordina toda la operación. Utiliza CuentaService, MovimientoService y NotificacionService para modificar las cuentas, registrar movimientos y generar notificaciones.	
	TransferenciaRepository — Guarda y consulta las transferencias mediante Spring Data JPA.	
notificacion	Notificacion, NotificacionController, NotificacionService	Guarda id, clabeCuenta, mensaje y fecha, consultables con GET /notificaciones?clabe=. Hoy se almacenan localmente; el código deja abierta la opción de usar RabbitMQ o Kafka, que no forma parte de la implementación actual.
	Notificacion — Entidad que almacena una notificación. Contiene id, clabeCuenta, mensaje y fecha.	
	NotificacionController — Expone GET /notificaciones?clabe=... para consultar las notificaciones asociadas a una cuenta.	
	NotificacionService — Genera y guarda las notificaciones a partir de los movimientos realizados. También escribe información en el log de la aplicación.	
	NotificacionRepository — Almacena y consulta las notificaciones mediante JPA	
spei	SpeiSolicitud, SpeiService, SpeiController, EstadoSpei	Administra solicitudes SPEI con estados PROCESANDO y ENVIADO. El campo idempotencyKey es único y evita procesar dos veces una misma operación.
	SpeiSolicitud — Entidad que representa una solicitud SPEI. Guarda id, idempotencyKey, claveOrigen, bancoDestino, claveDestino, monto, estado y creadoEn. La idempotencyKey es única en la base de datos.	
	SpeiController — Expone POST /spei. Recibe la solicitud y exige la cabecera Idempotency-Key. Si no se proporciona, devuelve un error 400.	
	SpeiService — Contiene la lógica para procesar las solicitudes SPEI y controlar la idempotencia. Comprueba si una solicitud con la misma clave ya existe.	
	SpeiTransacciones — Maneja las transacciones que reservan y ejecutan el SPEI. Utiliza una transacción independiente para reservar la idempotencyKey y otra para ejecutar el movimiento. También descuenta el saldo, registra el movimiento y genera la notificación.	
	SpeiSolicitudRepository — Busca y almacena solicitudes SPEI. Utiliza findByIdempotencyKey() para localizar una solicitud existente.	
	EstadoSpei — Define los estados posibles de una solicitud: PROCESANDO y ENVIADO.	

Lógica de idempotencia en `SpeiService`: si la `Idempotency-Key` no existe, se crea el SPEI, se ejecuta y queda en `ENVIADO`; si ya existe, se busca la operación, y si ya fue enviada se devuelve sin cobrar. Esto se comprobó con el SPEI de \$50.

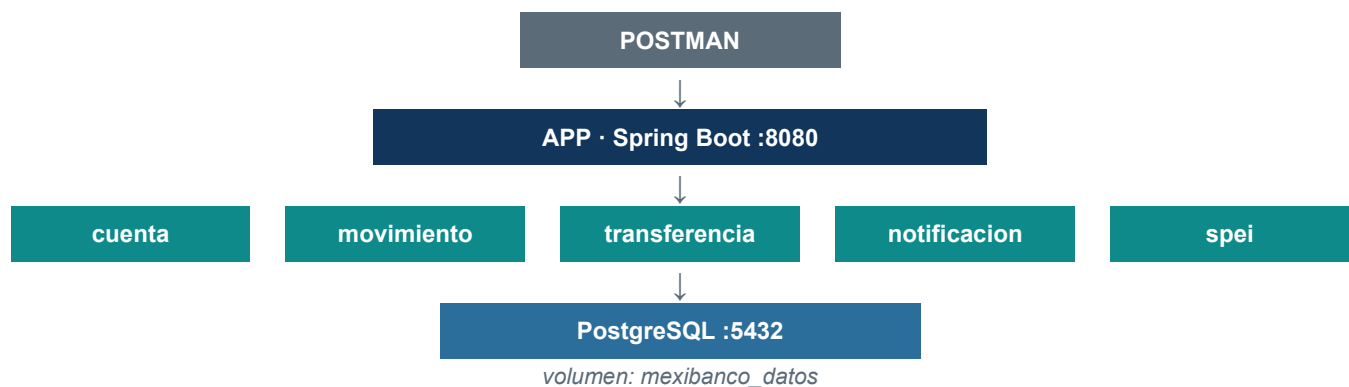
30 Análisis del docker-compose.yml

Elemento	Descripción
db	Usa <code>postgres:17-alpine</code> , recibe parámetros por variables de entorno y guarda su información en <code>mexibanco_datos</code> .
app	Construye su imagen con <code>build: .</code> y publica <code>8080:8080</code> , accesible en <code>http://localhost:8080</code> .
Comunicación	La app usa <code>DB_HOST=db</code> y <code>DB_PORT=5432</code> . El nombre del servicio funciona como host gracias a la resolución de nombres de Compose, sin IP fija.
depends_on	Con <code>condition: service_healthy</code> la app espera a que PostgreSQL esté saludable antes de iniciar.
Healthchecks	PostgreSQL usa <code>pg_isready</code> ; la aplicación usa <code>/actuator/health</code> .
Red y volumen	La red <code>mexibanco</code> conecta app y db. El volumen <code>mexibanco_datos</code> se monta en <code>/var/lib/postgresql/data</code> y conserva los datos aunque el contenedor se elimine.

31 Análisis del Dockerfile (multi-stage build)

Etapas 1 (`eclipse-temurin:21-jdk-alpine`): descarga dependencias, compila con `./mvnw clean package -DskipTests` y genera el JAR. **Etapas 2** (`eclipse-temurin:21-jre-alpine`): copia `target/*.jar` como `app.jar` y lo ejecuta con `java -jar app.jar`. Se usan dos etapas porque construir necesita herramientas (JDK, Maven) que no hacen falta para ejecutar; así se separan las herramientas de construcción del entorno final, que solo necesita el JRE.

32 Relación entre los componentes



La aplicación recibe las solicitudes desde Postman, procesa las reglas de negocio en sus módulos y almacena la información en PostgreSQL, cuyos datos persisten en el volumen.

33 Conclusiones

La práctica comprobó el funcionamiento de una aplicación bancaria ejecutada con Docker Compose: se crearon y consultaron cuentas, se hicieron transferencias y SPEI, se consultaron movimientos y notificaciones, y se rechazaron operaciones inválidas.

Idempotencia. Al enviar dos veces el mismo SPEI con la misma `Idempotency-Key`, el sistema no volvió a descontar el dinero y el saldo permaneció en \$750.

Manejo de errores. Los códigos HTTP reflejan el tipo de problema:

400	404	409	422
Datos o cabeceras inválidas	Recurso inexistente	Conflicto (CLABE duplicada)	Regla de negocio no permitida

Persistencia. Con `docker compose down` los contenedores desaparecieron pero los datos permanecieron en `mexibanco_datos`; con `docker compose down -v` también se eliminó el volumen y la cuenta devolvió 404. Así, los datos de PostgreSQL no dependen del ciclo de vida del contenedor cuando existe un volumen.

Arquitectura. El código separa los módulos `cuenta`, `movimiento`, `transferencia`, `notificacion` y `spei` bajo la estructura Controller, Service, Repository. El análisis de `docker-compose.yml` mostró cómo Docker administra servicios, red, healthchecks, variables de entorno y volumen, y el Dockerfile mostró el uso de un multi-stage build.

En conjunto, la práctica relacionó Docker, Docker Compose, Spring Boot y PostgreSQL con conceptos de sistemas distribuidos. La persistencia y la idempotencia fueron especialmente relevantes porque muestran cómo un sistema debe manejar tanto el almacenamiento de información como los reintentos de operaciones durante la comunicación entre sistemas.